

VectorMaton: Efficient Vector Search with Pattern Constraints via an Enhanced Suffix Automaton

Haoxuan Xie
haoxuan001@e.ntu.edu.sg
Nanyang Technological University
Singapore

Siqiang Luo
siqiang.luo@ntu.edu.sg
Nanyang Technological University
Singapore

ABSTRACT

Approximate nearest neighbor search (ANNS) has become a cornerstone in modern vector database systems. Given a query vector, ANNS retrieves the closest vectors from a set of base vectors. In real-world applications, vectors are often accompanied by additional information, such as sequences or attributes, motivating the need for fine-grained vector search with constraints on this auxiliary data. Existing methods support attribute-based filtering or range-based filtering on categorical and numerical attributes, but none supports predicates over sequence attributes. In relational databases, predicates such as LIKE and CONTAINS are fundamental operators for filtering pattern-matching records. As vector databases increasingly adopt SQL-style query interfaces, enabling such pattern predicates over sequence attributes (e.g., texts and biological sequences) alongside vector similarity search becomes essential. In this paper, we formulate a novel problem: given a set of vectors each associated with a sequence, retrieve the nearest vectors whose sequences contain a given query pattern. **Specifically, we mainly focus on the substring containment case.** To address this challenge, we propose VECTORMATON, an automaton-based index that integrates pattern filtering with efficient vector search. Extensive experiments on real-world datasets demonstrate that VECTORMATON consistently outperforms all baselines, achieving up to two orders of magnitude higher query throughput with similar or higher accuracy.

KEYWORDS

Vector database, constrained approximate nearest neighbor search

PVLDB Reference Format:

Haoxuan Xie and Siqiang Luo. VectorMaton: Efficient Vector Search with Pattern Constraints via an Enhanced Suffix Automaton. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/ForwardStar/VectorMaton>.

1 INTRODUCTION

The rapid advancement of deep learning has enabled the transformation of unstructured data (e.g., texts, images and graphs) into high-dimensional vector representations [8, 21, 30, 39], giving rise

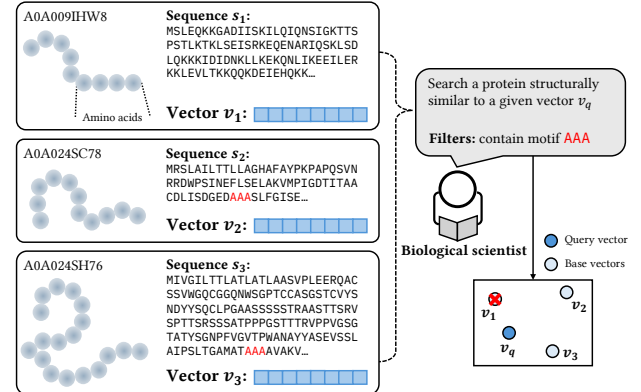


Figure 1: An example of pattern-constrained ANNS in biological database.

to vector databases as a new data management paradigm. In vector databases, vector search focuses on retrieving the most similar vectors according to a distance or similarity metric. **However, identifying the exact nearest neighbors in high-dimensional vector spaces is computationally expensive due to the curse of dimensionality [22].** As a result, approximate nearest neighbor search (ANNS) has emerged as the dominant approach and attracted significant attention from both academia and industry [5, 16, 19, 24, 26, 35, 36, 48].

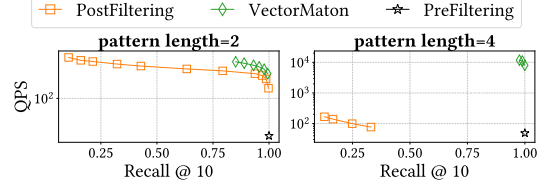
Motivation. In many real-world applications, vector embeddings are accompanied by associated sequences or other structured attributes. For example, in scientific document retrieval, the content of a paper can be embedded into a vector representation, while the paper title is naturally modeled as a sequence associated with the vector [43]. While recent studies have explored constrained approximate nearest neighbor search through *attribute-in* filtering [20, 32, 34, 38] and range-based filtering [25, 45, 53, 54, 57], these techniques primarily focus on numeric or categorical constraints. In contrast, **pattern constraints over sequences**, such as substring or motif matching, remain largely unexplored in the context of vector search. Given a query vector v and a pattern p , *ANNS with pattern constraints* aims to retrieve the nearest vectors whose associated sequences contain the pattern p . **Specifically, we mainly focus on substring containment queries (i.e., pattern is also a sequence).** The problem naturally arises in a wide range of real-world applications, including but not limited to the following examples:

- **Supporting LIKE and CONTAINS predicates in vector SQL.** In relational databases, predicates such as LIKE and CONTAINS are fundamental building blocks for SQL queries. These predicates largely involve substring queries, i.e., retrieving data that contains a specific pattern string. As vector databases increasingly adopt SQL-like query interfaces, supporting such pattern predicates alongside vector similarity search becomes essential. For example, in a document

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Methods	Short pattern		Long pattern		Index Size
	Efficiency	Quality	Efficiency	Quality	
PreFiltering	Low	Optimal	Moderate	Optimal	Small
PostFiltering	Moderate	Moderate	Moderate	Low	Small
OptQuery	High	High	High	High	$O(m^2)$
VECTORMATON	High	High	High	High	$O(m^{1.5})$

(a) Comparison of different methods (m is the total sequence length).



(b) Impact of pattern length.

Figure 2: Summary of challenges and methods.

database, users may not only want to retrieve documents with similar embeddings, but also require the documents to contain a specific keyword. While systems such as ElasticSearch [2] and PostgreSQL [1] have added support for hybrid vector similarity and LIKE predicate, they generally rely on either a filter-then-search or a search-then-filter execution strategy. These approaches treat vector search and pattern filtering as separate components and do not provide a unified index structure, leading to low query efficiency and quality. Efficiently integrating pattern constraints with approximate nearest neighbor search remains an open challenge. We would also like to remark that while we mainly focus on substring queries and do not consider the full SQL semantics, our approach can also be extended to support some more complex queries involving the underscore ‘_’ in query patterns in the LIKE predicate.

- **Similar protein retrieval with motif matching.** In biological databases, proteins and genomic sequences are increasingly represented by learned vector embeddings that capture functional or structural similarity (e.g., AlphaFold [27], ProtBERT [13]). For instance, the SwissProt [28] dataset contains proteins with a total sequence length reaching hundreds of millions of symbols. These vector embeddings are inherently tied to their underlying sequences, where patterns such as conserved motifs or binding sites play a critical role. For instance, in Figure 1, a biologist may wish to retrieve proteins that are functionally similar to a query protein (i.e., vector similarity) while containing a specific amino-acid motif (e.g., Cys-X₂-Cys) that is essential for metal binding [7]. Such queries require enforces both vector similarity and pattern constraints.
- **Semantic document retrieval with keyword filter.** In Retrieval-augmented generation (RAG) systems [31], documents are divided into chunks and represented as vectors for retrieval. While vector similarity captures semantic relatedness, users may impose keyword constraints (e.g., a specific person or country name) on the chunk text for fine-grained retrieval. For more details, we provide a case study of a concrete scenario for this usage in Section 6.5.

Challenge: Pattern constraints exhibit various selectivity and demand specialized index designs. In constrained vector search, different filtering conditions induce varying selectivity. Existing approaches to filtered approximate nearest neighbor search can be classified into three categories: *pre-filtering*, *post-filtering*, and *joint filtering* [32]. Pre-filtering first identifies qualifying vectors using an external index on vector-associated attributes and then performs a brute-force search over the filtered subset. However, when filter selectivity is high (i.e., when a large fraction of vectors qualify), the filtered subset becomes large, leading to substantial query overhead and poor scalability. Post-filtering, in contrast, first performs ANNS on the full vector index and subsequently filters the retrieved candidates. This approach is effective when selectivity is high enough that most nearest neighbors satisfy the constraint. However, under low-selectivity conditions, a large portion of retrieved candidates are discarded, resulting in degraded recall or increased query cost.

Unfortunately, **pattern constraints over sequences exhibit various selectivity**. Complex or long patterns typically match only a small number of sequences, leading to low selectivity, whereas short or simple patterns may match a large fraction of the dataset. As a result, neither pre-filtering nor post-filtering can robustly handle pattern-constrained queries across different selectivity. Joint filtering aims to integrate filtering conditions directly into the ANNS process, but typically requires specialized index designs tailored to specific types of constraints. Existing joint-filtering indices primarily target *attribute-in* and range-based predicates over categorical or numerical attributes. These methods are designed to support operators such as equality, subset inclusion, and numerical comparison. However, such techniques cannot be directly extended to pattern-based predicates over sequence attributes. Efficiently supporting *pattern constraints over sequences* within ANNS remains largely unexplored and poses a significant open challenge.

Table 2(a) shows the undesirable query trade-offs of PreFiltering and PostFiltering approaches. We conduct a case study in Figure 2(b) on the SIFT dataset [24] augmented with synthetic sequences of length 50. The system configuration and query generation are identical to those used in our main experiments (Section 6.1). We observe that, as the query pattern length increases, the performance of PostFiltering drops significantly, whereas PreFiltering consistently suffers from relatively low query efficiency.

Our solution. Our goal is to design an index that supports ANNS with *pattern constraints*, while provides query efficiency and result quality comparable to unconstrained vector search.

To optimize query efficiency and avoid unnecessary computations, a straightforward approach (OptQuery) is to query on the vector index containing only those vectors satisfying the given pattern constraint. This requires constructing the corresponding index for every possible pattern. However, the number of distinct substrings across all sequences is $O(m^2)$ in the worst case for a total sequence length m , leading to unaffordable space cost. While inverted file indexes [12, 29] reduce space consumption by avoiding materialization of all substrings, its query processing requires merging multiple posting lists that may involve duplicated vectors or false positives, leading to lower query performance.

To overcome these limitations, we present VECTORMATON, an automaton-based vector index for pattern-constrained ANNS. The key insight is that many patterns exhibit identical occurrence behavior across sequences. Such patterns can therefore be grouped and share the same index. For example, consider two sequences “ab”, “aab”. Then patterns “b” and “ab” are considered equivalent since they both occur at the end of each sequence. To realize this idea, we borrow the ideas from formal language theory, and leverage the *suffix automaton* (SAM) [10, 40]. While SAM has traditionally been used for substring matching, they are not designed for pattern-constrained vector search. In this work, we extend this technique

to both index a collection of sequences and integrate it with approximate nearest neighbor search. Since each pattern can still be directly associated with the vectors whose sequences contain it, the query efficiency remains comparable to OptQuery. Our resulting structure achieves efficient query processing while maintaining a worst-case space complexity of $O(m^{1.5})$, where m is the total sequence length. We further propose two space-saving strategies to reuse index across states to reduce redundancy and selectively construct index based on pattern selectivity to adapt to different query conditions. Together, these strategies enable a unified and scalable solution for pattern-constrained ANNS.

Empirically, we observe that the index size of VECTORMATON grows near-linear with respect to the total sequence length, while delivering substantial query performance improvements over pre- and post-filtering baselines. This scalability enables VECTORMATON to handle datasets with up to hundreds of millions sequence length.

Contributions. Our contributions are summarized as follows.

- We formally define the problem of ANNS with pattern constraints that, to the best of our knowledge, has not been systematically studied in prior work.
- We propose VECTORMATON, a novel index built on an enhanced suffix automaton that enables efficient pattern-constrained ANNS.
- We conduct extensive experimental evaluations on diverse real-world datasets, demonstrating that VECTORMATON consistently outperforms existing baselines, achieving up to two orders of magnitude higher query throughput at comparable accuracy.

2 PRELIMINARIES

2.1 Problem definition

Let $D = \{(v_1, s_1), (v_2, s_2), \dots, (v_n, s_n)\}$ be a dataset of size n , where v_i denotes the i -th vector and s_i denotes the sequence associated with the i -th vector. A sequence s can be a string, protein sequence, DNA sequence, etc. Let $m = \sum_{i=1}^n |s_i|$ be the total sequence length. Let V_p be the set of vectors whose associated sequences contain a pattern p . Pattern p is also a sequence, and a sequence s contains p if and only if p occurs as a consecutive subsequence (i.e., substring) in s . Given a query vector v_q , $V_{k,p}^*(v_q)$ is denoted as the top- k vectors in V_p closest to the query v_q . For an approximate set $V_{k,p}(v_q)$ of $V_{k,p}^*(v_q)$, we use *recall* to measure its quality, where recall is defined as $Recall(R) = \frac{|R \cap R^*|}{|R^*|}$ [25]. Our problem is defined as follows.

DEFINITION 1 (ANNS WITH PATTERN CONSTRAINTS). *Given a query vector v_q , a query pattern p , and an integer k , find a set of vectors $V_{k,p}(v_q)$ approximating the ground truth set $V_{k,p}^*(v_q)$.*

Our goal is to optimize the query throughput of ANNS with pattern constraints while maintaining a high recall. We remark that while the query pattern p , query vector v_q , sequences s_i , and vectors v_i may exhibit some degree of correlation in real-world applications, our goal is to develop a general solution whose correctness and effectiveness do not depend on such assumptions.

2.2 Hierarchical Navigable Small Worlds

The *Hierarchical Navigable Small Worlds* (HNSW) graph [36] is a widely adopted graph-based index for ANNS in high-dimensional vector spaces. In the following, we provide a brief overview of it.

HNSW structure. HNSW organizes vectors into a multi-layer graph. The bottom layer contains all vectors and provides dense

Notation	Description
n, m	the number and total length of all sequences
D	the dataset $\{(v_1, s_1), \dots, (v_n, s_n)\}$
V	base vectors $\{v_1, \dots, v_n\}$ in the dataset
S	sequences $\{s_1, \dots, s_n\}$ associated with the vectors
v_q, p	query vector v_q and query pattern p
V_p	base vectors whose associated sequences contain p
$V_{k,p}^*(v_q)$	top- k vectors in V_p closest to the query vector v_q
$V_{k,p}(v_q)$	approximated vector set of $V_{k,p}^*(v_q)$
M	the maximum degree in HNSW graph
ef_con, ef_search	candidate list capacity in HNSW construction/query
T	threshold of constructing HNSW graph in VECTORMATON

Table 1: Summary of notations.

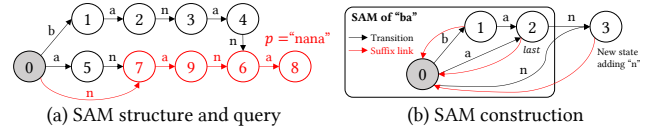


Figure 3: The SAM structure of the sequence “banana”.

local connections among nearby vectors, while higher layers contain progressively fewer nodes and act as long-range “highways”. The maximum degree of a node is limited by the hyperparameter M . Consequently, the overall space complexity of HNSW is linear in the number of indexed vectors.

Query processing in HNSW. To answer a query of top- k nearest vectors, HNSW performs a greedy search starting from an entry point at the top layer, iteratively moving to neighboring nodes that are closer to the query vector until we find the approximated nearest one, and uses it as the entry node of the next layer. After descending to the bottom layer, HNSW executes a bounded best-first search to find a set of nearest vectors and selects the top- k as the results. The trade-off between query latency and recall is controlled by the parameter ef_search , which specifies the maximum size of the candidate list maintained during the bottom-layer search and is similar to its construction parameter ef_con . Larger ef_search increase recall at the cost of higher query latency, while smaller ef_search favor faster queries with reduced accuracy.

2.3 Suffix automaton

The *suffix automaton* (SAM) [10, 11] is a compact deterministic finite automaton that represents all suffixes of a given sequence. The general suffix automaton (GSA) further extends SAM to handle a collection of multiple sequences. Since a CONTAINS predicate checks whether a pattern p appears as a substring, it can be equivalently viewed as checking whether p is a prefix of at least one suffix of the sequence. Therefore, both SAM and GSA naturally support efficient verification of whether a pattern is contained in a single sequence or in a sequence collection.

Overview of SAM. A SAM is represented as a directed acyclic graph whose nodes, called *states*, compactly represent groups of substrings with shared characteristics. Directed edges, referred to as *transitions*, are labeled with single characters and indicate how substrings can be extended. SAM has a unique source node called the *initial state*, from which all other states are reachable. There is a one-to-one mapping between substrings of the sequence and transition paths starting from the initial state. While different substrings may share prefixes or terminate at the same state, SAM compactly merges these overlaps. This design enables SAM to represent all substrings in linear space, while allowing pattern checks to be performed by a single left-to-right traversal of the automaton.

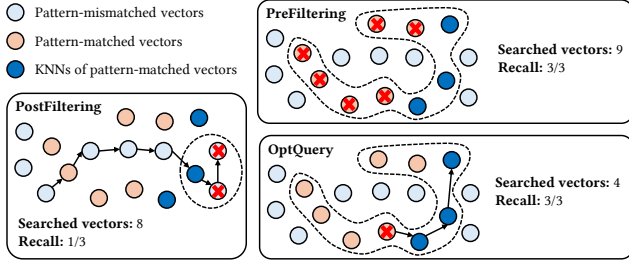


Figure 4: Comparison of different baselines.

EXAMPLE 1. Figure 3(a) shows the SAM constructed for the sequence “banana”, where state 0 is the initial state. Given the pattern $p = \text{“nana”}$, we start from state 0 and follow transitions labeled n, a, n, and a. Since the traversal successfully reaches a state in the end, pattern p is contained in the sequence “banana”.

SAM construction. The SAM of a sequence is built incrementally with one character at a time. Let *last* denote the most recently created state corresponding to the full prefix constructed so far. When a new character c arrives, we create a new state and add a transition labeled c from *last*. This extends the suffixes in *last* to end at the new position. We then follow a small chain of *suffix links* [10] from *last* to update earlier states that should also gain a transition labeled c . These suffix links connect states representing substrings with shared suffixes. This incremental procedure constructs the SAM in $O(|s|)$ time for a sequence s .

EXAMPLE 2. Figure 3(b) illustrates a partial construction of the SAM for the sequence “banana”. After processing the prefix “ba”, state 2 is the current last state and represents the suffixes “ba” and “a”. When the next character ‘n’ is added, these suffixes are extended to “ban” and “an”, which are captured by a new state 3. In addition, the single-character suffix “n” is also introduced. To account for this new suffix, the construction follows the suffix link of state 2 and adds a transition labeled ‘n’ from the initial state 0 to state 3.

3 PROPOSED BASELINES

3.1 Optimal query baseline for HNSW

We begin by proposing a fundamental question in solving the pattern-constrained ANNS problem in Definition 1: *under the HNSW query framework, what is the best achievable trade-off between query efficiency and recall?*

An intuitive solution (OptQuery) is obtained by decoupling pattern filtering from vector search. Specifically, let $V_p \subseteq V$ denote the subset of items whose associated sequences contain pattern p . We first identify V_p , and then build and query an HNSW index over *only* these qualified items. Running a standard HNSW search on this filtered graph avoids any exploration of disqualified items without affecting the accuracy. Therefore, this baseline requires enumerating all patterns that appear in the dataset and building an independent HNSW graph for each pattern p over the filtered vector set V_p . Specifically, for each sequence s_i , we extract all of its substrings and insert the associated vector v_i into the HNSW graph corresponding to each substring pattern. At query time, we first check whether the queried pattern exists in the index. If so, we directly perform an HNSW search on the graph associated with that pattern; otherwise, no feasible result exists and the query terminates immediately. As each sequence s_i contains $O(s_i^2)$ substrings, the total space required for OptQuery is bounded by $O(\sum_i s_i^2) = O(m^2)$.

3.2 PreFiltering and PostFiltering

Algorithm 1 shows the query processes of *PreFiltering* and *PostFiltering*. Given the query vector v_q , query pattern p and integer k , PreFiltering first identifies candidate vectors V_p and then performs a brute-force search over this filtered subset to find k nearest neighbors to v_q . This approach guarantees exact results but often incurs significantly higher query time due to the exhaustive search. PostFiltering, in contrast, performs a standard ANN search over the full HNSW index and subsequently filters the results to retain only those satisfying the pattern constraint. While PostFiltering achieves query efficiency comparable to HNSW-based methods such as the optimal baseline, it may sacrifice recall because some relevant vectors can be missed during the initial ANN search.

We further introduce a hybrid strategy that dynamically switches between PreFiltering and PostFiltering according to the query selectivity. Specifically, since PostFiltering retains only the satisfying vectors among the top- ef_search ANN results, the expected number of valid results is approximately $ef_search \cdot s$, where s denotes the selectivity (i.e., the fraction of sequences satisfying the query pattern). Therefore, we use $\frac{k}{ef_search}$ as the selectivity threshold. When $s < \frac{k}{ef_search}$, the expected number of satisfying results returned by PostFiltering is less than $ef_search \cdot \frac{k}{ef_search} = k$, which can significantly degrade recall. In this case, we apply PreFiltering to guarantee sufficient candidate vectors. Otherwise, we apply PostFiltering to achieve higher query efficiency.

Algorithm 1: Pre/Post-Filtering

- 1 **Function** PreFiltering(v_q, p, k):
 - 2 Identify V_p from an index of S ;
 - 3 Return k closest vectors in V_p ;
 - 4 **Function** PostFiltering(v_q, p, k, ef_search):
 - 5 Search in the HNSW graph $H(p)$ with parameters v_q, k, ef_search and return ef_search results;
 - 6 Return k closest vectors whose associated sequences contain p ;
-

Inverted indexes (e.g., BM25 [46]) are widely used for textual retrieval and can be adapted as an approximate filtering mechanism for pattern-constrained vector search. Therefore, we consider an additional hybrid approach, denoted as *BM25Filtering*, which combines an inverted index with HNSW search. Specifically, BM25Filtering first builds an inverted index over all 2-grams appearing in the sequence collection. For each 2-gram, an HNSW graph is constructed over the vectors whose associated sequences contain that 2-gram. Given a query pattern p , the algorithm decomposes p into its constituent 2-grams and queries the corresponding HNSW graphs to retrieve candidate vectors. The retrieved candidates are then ranked according to their BM25 scores with respect to p , and the top-ranked candidates are further sorted by vector distance to produce the final top- k results. However, we would like to remark that BM25 (or other inverted index methods) is designed for textual relevance rather than exact pattern matching. Consequently, sequences with high BM25 scores may not actually contain the query pattern. Even if a sequence contains the query pattern, it may still be filtered out during the BM25 ranking process as it contains less 2-grams than other candidate sequences. Moreover, each queried HNSW graph contains vectors associated with a particular 2-gram rather than

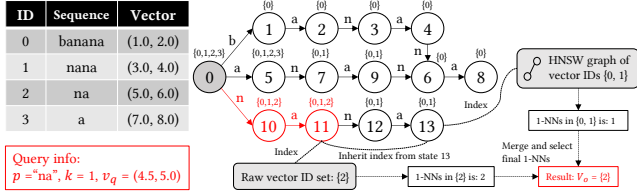


Figure 5: An example VECTORMATON index and query.

the complete pattern, introducing a large number of false-positive candidates that harm the query performance.

3.3 Open challenges and opportunities

Figure 4 summarizes the three baseline approaches. PreFiltering first identifies all pattern-matched vectors and then performs vector search over the filtered subset. When the pattern selectivity is low, this leads to examining a large number of candidates and consequently low query efficiency. PostFiltering, on the other hand, performs ANN search before applying the pattern constraint. As a result, many retrieved candidates may not satisfy the pattern, which degrades recall or requires expanding the search space to compensate. OptQuery achieves the best efficiency-recall trade-off by constructing vector indexes for pattern-matched subsets, ensuring that only relevant vectors are examined during search. However, this strategy incurs a quadratic space cost, as it potentially requires building indexes for $O(m^2)$ distinct patterns, making it impractical for large-scale datasets.

The key challenge is therefore how to retain the query performance of OptQuery while reducing its index size. Our central insight is that although there can be $O(m^2)$ possible substrings in a sequence collection, many of them share identical occurrence behavior and can be grouped into $O(m)$ equivalence classes via an automaton structure. By constructing indexes over these equivalence classes rather than individual patterns, we can significantly reduce the index size while preserving efficient query processing.

4 VECTORMATON

We now present VECTORMATON, an index designed to support efficient approximate nearest neighbor search under pattern constraints. We first describe the overall index structure in Section 4.1. As illustrated in Figure 5, VECTORMATON consists of an enhanced suffix automaton (ESAM) that indexes all substrings of the sequences, with each state augmented by either a HNSW graph or a raw vector ID set. To process a query pattern p , we traverse ESAM along edges labeled with the symbols of p until reaching the corresponding state. From this state, we perform a k -nearest neighbor (k -NN) search over the vectors in its own index as well as the index inherited from one of its successor state. The results from these searches are then merged to produce the final top- k nearest neighbors. We then detail the query processing and index construction procedures in Sections 4.2 and 4.3, respectively, and the index maintenance scheme in Section 4.4.

4.1 Index structure

Figure 5 illustrates an overview of the VECTORMATON index structure. We propose an enhanced suffix automaton, termed as ESAM, which consists of states and transitions. Each state is associated with a corresponding state index either as an HNSW graph or as a raw vector ID set.

ESAM states. The proposed VECTORMATON index constructs an enhanced suffix automaton over all sequences in the collection.

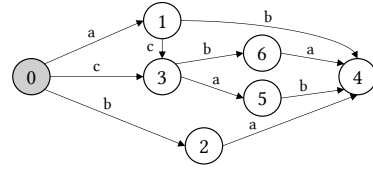


Figure 6: The general suffix automaton [40] of sequences ac, acab and acba.

Recall that the general suffix automaton (GSA) [40] compactly represents all substrings of a sequence collection and supports efficient *pattern existence* queries (see Section 2.3). However, GSA cannot be directly extended to determine *which sequences* contain a given pattern. This limitation arises because a single automaton state may represent multiple substrings whose occurrences span different subsets of sequences (see Example 3).

EXAMPLE 3. Figure 6 illustrates a GSA built over the sequences ac, acab, and acba. Consider two patterns, $p_1 = cba$ and $p_2 = cab$. Although the GSA correctly determines the existence of both patterns by ending in the same state (state 4), their occurrence sets differ: p_1 occurs only in sequence acba, whereas p_2 occurs only in sequence acab. Consequently, the GSA cannot distinguish which sequences contain a given pattern.

To support pattern-constrained queries, ESAM redefines the semantics of automaton states: each state corresponds to a set of *equivalent patterns* and is associated with the set of vector IDs whose underlying sequences contain all patterns represented by that state. This refinement ensures the correctness of answering our problem. In the following, we formalize the definition of equivalence class.

DEFINITION 2 (POSITION LIST (POSTLIST)). Given a pattern p that appears in the sequence collection, we define its position list, denoted as $poslist(p)$, as the set of pairs (id, pos) , where id is the identifier of a sequence containing p , and pos is the end position of an occurrence of p in that sequence.

EXAMPLE 4. Consider the pattern $p = anan$, which corresponds to state 6 in Figure 5. The pattern occurs only once, in the sequence banana with sequence ID 1, ending at position 4 of the sequence. Therefore, $poslist(p) = \{(1, 4)\}$.

DEFINITION 3 (EQUIVALENCE CLASS). Two patterns p_1 and p_2 are considered equivalent if and only if $poslist(p_1) = poslist(p_2)$.

DEFINITION 4 (MAXIMAL PATTERN). Given a pattern p that appears in the sequence collection, we say p is maximal if and only if p is longest within its equivalence class.

As a result, the patterns of each equivalence class corresponds to only one set of vector IDs, which is the union of IDs in its position list. Moreover, it is straightforward to observe that each equivalence class contains exactly one maximal pattern. In particular, if two maximal patterns have the same length and are equivalent, their occurrences in the sequences are thus identical, and extracting the pattern from their occurrences yields the same resulting pattern.

Since each state in ESAM represents a set of equivalent patterns, the number of states is equal to the number of maximal patterns in the sequence collection. Consequently, bounding the number of maximal patterns directly yields a bound on the number of states. Due to space limit, for all following lemmas and theorems, we defer their proofs to the appendix of our extended version [52].

LEMMA 1. Let S be a collection of sequences with total length m . The number of maximal patterns in S is bounded by $O(m)$. Therefore, the number of states in ESAM is also bounded by $O(m)$.

The linear bound on the number of states is a fundamental property of suffix automata. Lemma 1 demonstrates that our ESAM preserves this property, avoiding the potential for substantial space blowup. In the following, we further establish an upper bound on the total size of the ID sets associated with all states.

LEMMA 2. *The total size of all associated ID sets in ESAM is bounded by $O(m^{1.5})$, where m is the total length of all sequences.*

ESAM transitions. As we have defined the states of ESAM, we next describe how transitions between states are established. As discussed in Section 2.3, transitions in classical suffix automata correspond to extending a pattern by one character. This principle naturally generalizes to our multi-sequence setting. Specifically, the following lemma formalizes the correctness of the resulting transition construction.

LEMMA 3. *Let A and B be two equivalence classes, corresponding to states A and B . If there exist patterns $p_A \in A$ and $p_B \in B$ such that $p_B = p_A \cdot c$ for some character c , then for every other pattern $p'_A \in A$, the extended pattern $p'_A \cdot c$ also belongs to B . Consequently, a transition labeled c can be established from state A to state B .*

Since each transition strictly increases all position values pos by one, the states and transitions in ESAM form a directed acyclic graph (DAG). Moreover, for any transition $i \rightarrow j$, the associated vector ID sets satisfy $V_j \subseteq V_i$. This monotonicity property enables effective reuse of index information between a state and its descendant states.

State index. While each state in ESAM is associated with a vector ID set, as illustrated in Figure 5, these sets exhibit substantial overlap across states. Moreover, different states correspond to patterns with varying selectivity. Consequently, constructing an independent HNSW index for every state would be highly space-inefficient. To address this issue, we propose two space-optimization strategies: **(1) an index reuse strategy** and **(2) a skip-build strategy**.

For index reuse strategy, let V_j be the associated ID set over state j , and I_j be the ID set to index associated with state j that contains $|I_j|$ vectors. Among all descendants of j (i.e., states reachable from j via a directed path), let k be the descendant with the largest indexed ID set. We call state k the *inherited state* of state j . Instead of building I_j over the entire set V_j , we construct I_j only over the difference set $V_j \setminus I_k$. During query processing on state j , we independently retrieve the k -NNs from both I_j and I_k , and then merge the two result sets. This reuse strategy achieves the comparable query accuracy as an index built directly over V_j while reduces the total space consumption.

For skip-build strategy, as states may have widely varying ID set sizes due to differences in pattern selectivity, constructing a full HNSW graph for every state is often unnecessary. Under the skip-build strategy, when the size of a state’s ID set is below a threshold T , we store only the raw vector ID set and perform brute-force search when querying. This approach is generally more efficient than graph-based search for very small datasets. For example, when the search parameter ef_search exceeds the graph size, HNSW search may degenerate into an exhaustive traversal of all vertices, which is typically slower than direct brute-force evaluation.

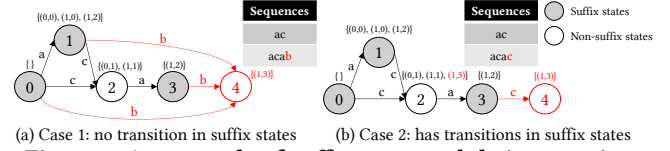


Figure 7: An example of suffix states and their extension.

4.2 Query processing

To answer a query, we first leverage the ESAM to locate the state corresponding to the query pattern, which is associated with the filtered vector set. We then perform k -NN search over the index associated with this state and, if applicable, its inherited index.

Algorithm 2 presents the details. The algorithm begins at the initial state 0. It sequentially scans the query pattern and follows the corresponding transitions in ESAM until either the entire pattern is consumed or no valid transition exists (lines 23-25). Let cur denote the resulting state. The algorithm then retrieves the index associated with state cur as well as the index of its inherited state, if such a state exists (lines 26-27). Finally, it independently performs k -nearest neighbor searches on the two indexes and merges the results to obtain the final answer set (lines 28-29).

As VECTORMATON adopts a skip-build strategy that adaptively constructs each index either as an HNSW graph or as a raw vector ID set, the query processing is different on the two types of index. If an index is implemented as an HNSW graph, we apply the standard HNSW search procedure with parameter ef_search . Otherwise, we perform a brute-force scan over all associated vectors and select the top- k nearest neighbors.

As the indexes of a state and its inherited state together form an exact cover of the state’s associated vector set, merging the top- k results returned from these two indexes is **lossless**. Consequently, the resulting recall is comparable to that of OptQuery, which queries over the entire satisfying vector set V_p .

LEMMA 4. *Let V_j be the associated ID set over state j , and I_j be the ID set of the index (HNSW or raw ID set) of state j . Then for each state j and its inherited state k , the sets I_j, I_k form an exact cover of V_j , i.e., $I_j \cup I_k = V_j$ and $I_j \cap I_k = \emptyset$.*

4.3 Index construction

In this section, we introduce how to construct the ESAM and state index, respectively. Additionally, we provide a parallel construction approach that accelerates the index construction process.

ESAM construction. For ESAM construction, we propose a generalized variant of the SAM construction described in Section 2.3. We process the sequence collection incrementally, and for each sequence, we process its symbols in order. Next, we aim to identify new states and transitions upon processing each symbol.

The key question of is **which old states may give rise to new states**. Since symbols are processed incrementally, each newly read symbol extends all suffixes of the currently processed sequence by one character, potentially forming new equivalence classes. Consequently, the construction must maintain the states that correspond to suffixes of the current sequence.

DEFINITION 5 (SUFFIX STATE). *Consider the sequence s_i currently being processed and suppose the algorithm has processed position j . Upon reading the next symbol (at position $j + 1$), a state (i.e., an*

Algorithm 2: VECTORMATON

```

1 Function Build( $V, S, T$ ):
  // Build ESAM via suffix-link extension
2  Initialize ESAM with state 0;
3  foreach sequence  $s_i \in S$  do
4     $last \leftarrow 0$ ;
5    foreach symbol  $c$  in  $s_i$  do
6      Create new state  $cur$ , follow suffix links from  $last$  and:
      (i) connect missing  $c$ -transitions to  $cur$ ; (ii) stop at
      first state  $A$  with  $c$ -transition;
7      if no such state  $A$  exists then  $link(cur) \leftarrow 0$ ;
8      else
9         $B \leftarrow$  the  $c$ -successor of  $A$ ;
10       if  $|B| = |A| + 1$  then  $link(cur) \leftarrow B$ ;
11       else
12         Clone  $B$  as  $B'$  and redirect transitions;
13          $link(cur) \leftarrow B'$ ;
14      $last \leftarrow cur$ ;
  // ID propagation
15  Follow suffix link from  $last$  and (1) add ID  $i$  to their ID
  set; (2) stop at first state that already contains  $i$ ;

  // Build state indexes
16  foreach state  $u$  in reverse topological order do
17     $v \leftarrow$  descendant with largest indexed set;
18     $base(u) \leftarrow ID(u)$  excluding the indexed set of  $v$ ;
19    if  $|base(u)| < T$  then Store  $I(u) \leftarrow$  raw ID set of
     $base(u)$ ;
20    else Build index  $I(u) \leftarrow$  HNSW over  $base(u)$ ;

21 Function Query( $v_q, p, k, ef\_search$ ):
22   $cur \leftarrow$  the state 0;
23  for  $i \leftarrow 0, 1, \dots, |p| - 1$  do
24    if no transition  $p_i$  in  $cur$  then return 0;
25     $cur \leftarrow$  next state of transition  $p_i$ ;
26   $next \leftarrow$  the state that  $cur$  inherits index from;
27   $I(cur), I(next) \leftarrow$  the index of  $cur, next$ ;
28   $res_1, res_2 \leftarrow$  the  $k$ -NN results of  $v_q$  from  $I(cur), I(next)$ ;
29  return the top- $k$  closest vector IDs in  $res_1 \cup res_2$ ;

```

equivalence class) is called a suffix state if and only if (1) its position list is empty, or (2) its position list contains (i, j) .

EXAMPLE 5. Figure 7(a) shows an example of suffix states. Before extending symbol b , states 0, 1, 3 are suffix states as their position list is either empty or contains $(1, 2)$.

The next question is **how to extend these suffix states to new states**. By Lemma 3, for a suffix state A with position list $poslist(A) = \{(x_k, y_k)\}$, extending A by the next symbol $c = s_i[j + 1]$ yields a state whose position list is $\{(x_k, y_k + 1) | s_{x_k}[y_k + 1] = c\}$. Consequently, if a suffix state A has no outgoing transition labeled c prior to the extension, none of its existing occurrences can be extended by c , and the extension introduces only the new occurrence ending at $(i, j + 1)$. Therefore, all such suffix states point to the same newly created state whose position list is exactly $\{(i, j + 1)\}$.

LEMMA 5. Let $c = s_i[j + 1]$ be the next symbol to be processed. For all suffix states A that do not have an outgoing transition labeled c , extending A by c leads to the same state.

EXAMPLE 6. Figure 7(a) shows an example when suffix states have no corresponding transition 'b'. We create a new state and connect these suffix states to the new state with position list $\{(1, 3)\}$.

Conversely, suppose a suffix state A already has an outgoing transition labeled c prior to the extension, and let B denote its target state. We examine whether all occurrences represented by B remain valid after introducing the new occurrence $(i, j + 1)$. If adding $(i, j + 1)$ preserves the equivalence class represented by B , then no structural change is required: B simply augments its position list with $(i, j + 1)$ and continues to represent the updated class. Otherwise, the extension violates the equivalence of occurrences in B . In this case, we split the class by creating a copy B' . The original state B retains the occurrences that cannot be extended to $(i, j + 1)$, while B' represents the refined equivalence class that includes $(i, j + 1)$, and then replace connection $A \rightarrow B$ by $A \rightarrow B'$.

EXAMPLE 7. Figure 7(b) illustrates the case where suffix states 0 and 1 both have an outgoing transition labeled 'c'. Since all occurrences represented by state 2 remain consistent after adding the new occurrence $(1, 3)$, no structural change is required.

However, directly scanning all suffix states would be inefficient. Thus we utilize the *suffix link* mechanism [10] to provide an efficient way to process them. While the concept is primarily designed for single-sequence scenario (see Section 2.3), it can be naturally generalized to our multi-sequence ESAM. For space reasons, the detailed correctness proof is deferred to the appendix of the extended version [52]. In the following, we outline the ESAM construction process via suffix links (Algorithm 2). Specifically, the suffix states form a chain ordered by decreasing maximal pattern length via suffix links, which splits into at most two contiguous parts: states without an outgoing transition labeled c , and states with such a transition. For states without transition c , we create a new state and connect all of them to it via transitions labeled c (lines 5-6). For states with transition c , let A be the first such state and let B be its c -successor (line 9). If the maximal pattern length of B equals that of A plus one, then B already represents the correct equivalence class after extension and no structural change is required (line 10). Otherwise, we create a copy B' of B so that B' represents position list extendable by the new symbol (i.e., $poslist(B') = poslist(B) \cup \{(i, j + 1)\}$), while B preserves the original position list (lines 12-13). We then update the remaining suffix states on the chain by redirecting their c -transitions from B to B' . Consequently, each extension step introduces at most two new states and updates transitions only along the suffix-link chain, yielding amortized constant processing time. The suffix links themselves can also be maintained in constant time per symbol. After processing a sequence, we need to reset suffix state to 0 before processing the next sequence.

Vector ID propagation. Vector IDs can be propagated in an on-line manner during ESAM construction. Since each extension step processes all suffix states along the suffix-link chain, these states correspond exactly to patterns that end at the current position and should therefore include the current sequence ID. Thus, we propagate the vector ID by traversing the suffix-link chain starting from the newly created state and adding the ID until reaching a state that already contains it (line 15 of Algorithm 2).

State index construction. To address the significant overlap among ID sets and handle small ID sets, we propose two strategies: an **index reuse strategy** and a **skip-build strategy**. Specifically, we construct state indexes in reverse topological order of ESAM. In this order, when processing a state u , all its descendant states have already been processed. We identify the descendant of u with the largest indexed ID set in constant time. We then define the *base set* of u as the difference set between $ID(u)$ and the largest indexed set of its descendants. If $|base(u)| < T$, we directly store $base(u)$ as a raw ID set. Otherwise, we build an HNSW index over the vectors corresponding to $base(u)$. This process ensures effective index reuse while avoiding unnecessary graph construction for small sets. The detailed procedure is shown in lines 16–20 of Algorithm 2.

We remark that all vectors are stored in a global array. Each HNSW graph maintains only the IDs of vectors rather than storing local copies of the vectors themselves. Therefore, the total space usage of VECTORMATON is dominated by the storage of ID sets and ESAM structures, and is linear in the total size of all ID sets.

THEOREM 1. *Let m denote the total length of all sequences. With a constant size of symbol set, the overall space complexity of VECTORMATON is bounded by $O(m^{1.5})$.*

Parallel construction. The primary bottleneck in index construction is building the HNSW graphs. Although the construction of a single HNSW graph is inherently sequential and difficult to parallelize efficiently, different state indexes are independent once their descendant states have been processed. To exploit this property, in parallel settings, we maintain a concurrent queue that stores states whose descendant states have all been processed. Worker threads repeatedly dequeue these states, construct their corresponding indexes (either raw sets or HNSW graphs), and enqueue their predecessor states if appropriate. This design enables parallel construction of multiple HNSW graphs to improve construction efficiency while preserving correctness. Our experimental results show that with 16 threads, the index can be constructed within 2 hours on datasets with hundreds of millions sequence length.

4.4 Index maintenance

Insertion. Since ESAM is constructed incrementally, it can naturally support insertion (lines 3-16 of Algorithm 2). The main challenge lies in maintaining the index reuse strategy while preserving the correctness of Lemma 4. Recall that, under the index reuse strategy, each ESAM state may inherit the index of one of its descendant states such that the local index and the inherited index together form an exact cover of the state-associated data. Failure to maintain this property may result in increased index size and degraded query performance. During static construction, the exact-cover property is established through a reverse topological traversal after all data has been inserted (lines 17-21 of Algorithm 2). However, in a dynamic setting, repeating such traversals after every insertion would be prohibitively expensive.

We propose a lightweight online index update with no additional asymptotic time overhead while maintaining the correctness of Lemma 4. Specifically, when inserting a new sequence-vector pair, we first identify all affected states that should contain the new data, as well as any newly created states generated during the ESAM update process. We then update the corresponding indexes of all these states according to the following rules:

- For a newly created state, we construct a local index over all vectors associated with that state directly.
- For an existing affected state without an inherited index, we directly insert the new vector into its local index.
- For an existing affected state with an inherited index, we recursively process its inherited state, until (1) it does not have an inherited index, or (2) its next inherited state is processed before or not affected. Then, starting from the final state reversely, we check whether the new vector is already inserted into its inherited index. If so, no update to the local index is required; otherwise, the vector is inserted into the local HNSW index.

This procedure does not introduce additional asymptotic overhead beyond the ESAM update process, since each affected state and newly created state is processed exactly once.

We next show that the exact-cover property of Lemma 4 is preserved. The correctness of the first two cases is straightforward. For the third case, let I_j denote the local index of a state and I_k denote its inherited index. By the above rules, when we process I_j , the newly inserted vector is added to I_j if and only if it is not already covered by I_k . Therefore, the union $I_j \cup I_k$ must contain this new vector and covers all data associated with the state. It remains to show that $I_j \cap I_k = \emptyset$. Suppose, for contradiction, that both I_j and I_k contain the newly inserted vector. Then the insertion into I_j must have occurred without observing the vector in I_k . However, updates are propagated recursively through inherited indexes, implying that I_k is always updated before I_j performs its coverage check. Consequently, I_j would detect that the vector is already covered by I_k and would not insert it, yielding a contradiction. Therefore, $I_j \cap I_k = \emptyset$, and Lemma 4 is preserved after the insertion.

Deletion. Deletion is more challenging, as both ESAM and the HNSW index do not naturally support efficient structural removal. Instead of modifying the structure, we discuss a lazy-deletion strategy to mark the deleted data and ignore them during query processing, which is further described in the appendix of our extended version [52]. We remark that designing an efficient garbage collection mechanism to reclaim space from deleted entries remains challenging and is a direction for future work.

5 DISCUSSIONS

The pattern constraint considered in this paper primarily corresponds to substring matching, as defined in Section 2.1. Nevertheless, VECTORMATON can be simply extended to support several more expressive pattern predicates.

The underscore ‘_’ in the SQL LIKE predicate. We first consider substring matching with the wildcard symbol ‘_’, which in SQL LIKE predicates represents an arbitrary single symbol. To support such queries, when VECTORMATON encounters a ‘_’ during pattern traversal, it explores all outgoing transitions from the current state instead of following a single transition corresponding to a specific symbol. Each transition corresponds to an independent search branch. After processing the remaining pattern symbols, the results from all branches are merged and ranked according to vector distance, and the top- k closest vectors are returned.

EXAMPLE 8. *In Figure 5, consider the pattern $p = _a$. Starting from state 0, the first symbol is a wildcard, so all three outgoing transitions labeled ‘b’, ‘a’, and ‘n’ are explored, reaching states 1, 5, and 10, respectively. The second symbol is ‘a’. Since state 5 has*

Datasets	No. vectors	Total seq. len.	Dim.
spam [47]	489	13,643	384
words [15]	8,000	56,209	3,072
mtg [49]	21,550	1,504,633	1,152
arxiv [37]	157,605	9,851,413	768
prot [28]	455,692	116,326,099	1,024
code [23]	1,838,414	40,940,167	768

Table 2: Datasets used in our experiments.

no outgoing transition labeled ‘a’, this branch terminates without producing any result. In contrast, the branches through states 1 and 10 continue to states 2 and 11, respectively, where the corresponding ANN searches are performed. Finally, the results from the two valid branches are merged, and the top-k closest vectors are returned.

We note that the number of outgoing transitions from each state is bounded by the alphabet size, which is typically a small constant. Moreover, not all traversal branches necessarily reach a valid state, as some may terminate early due to missing transitions. Therefore, when the query pattern contains only a small number of wildcard symbols ‘_’, the number of additional HNSW searches remains limited. Given the high query efficiency of VECTORMATON, we expect this extension to be practical in real-world workloads and to introduce only a modest query overhead.

Disjunctive OR queries. For a disjunctive query containing multiple patterns, VECTORMATON executes the query process independently for each pattern, merges the resulting candidate sets, and returns the top-k vectors ranked by vector distance. Since the individual pattern queries are independent, they can be processed in parallel. We remark that VECTORMATON achieves high query efficiency for individual pattern queries. Consequently, this will not introduce a large additional overhead when the number of disjunctive patterns is modest.

6 EXPERIMENTS

6.1 Setup

Hardware and datasets. All experiments were conducted on a machine equipped with an AMD Ryzen Threadripper PRO 7975WX 32-core processor and 755 GB of RAM. The code was compiled using GCC 11.4.0 on Ubuntu Linux with the -O3 optimization flag enabled. We evaluate all methods on a diverse set of datasets collected from Hugging Face and the SpamAssassin corpus, as summarized in Table 2. For space limit, detailed description of each dataset is deferred to the appendix of our extended version [52].

Baselines. We compare VECTORMATON against **OptQuery**, **PreFiltering**, **PostFiltering**, **Hybrid**, **BM25Filtering**, **ACORN-1** [44], **ACORN-γ** [44], **pgvector** [1] and **ElasticSearch** [2]. For PreFiltering, we first use the enhanced suffix automaton as the filtering index S to retrieve candidate vectors, and then perform vector search over the filtered subset. Hybrid is the strategy combining PreFiltering and PostFiltering as described in Section 3.2. ACORN-1 and ACORN-γ are general solutions for constrained vector search, and we incorporate our enhanced suffix automaton to build their filter map (which is required in query processing). Pgvector is a vector extension of PostgreSQL, which supports hybrid SQL queries with vector similarity based on HNSW index and LIKE predicates. To minimize disk I/O overhead and ensure fair comparison, we store all data in temporary tables and configure temp_buffers to 128 GB so that query processing is performed entirely in memory.

ElasticSearch is an engine that supports hybrid queries by combining vector similarity based on HNSW index with keyword-based filtering (e.g., match or wildcard predicates). To minimize disk I/O overhead and ensure fair comparison, we allocate sufficient JVM heap space to keep all vectors and indices in memory. We remark that pgvector and ElasticSearch are implemented as a database system, which may introduce additional overhead compared to our native implementations over the queries.

Parameter settings. PreFiltering does not involve any ANN-specific tuning parameters. For VECTORMATON, we vary the HNSW search parameter ef_search in the range [8, 1024]. The maximum supported ef_search of pgvector is 1000 so we configure its range as [8, 1000]. For other methods, we apply a larger range for ef_search as [8, 4096] to improve their recall. ElasticSearch does not support adjusting ef_search , and we adjust its parameter “num_candidates” alternatively. The maximum degree of the HNSW graph is set to $M = 16$, and the construction parameter ef_con is set to 200. ACORN requires larger M to ensure connectivity in its graph structure, and hence its parameter M is set as 32. For VECTORMATON, the threshold T of constructing a HNSW graph is set to 800.

Queries. We generate 1,000 queries for each pattern length $|p| \in \{2, 3, 4, 5, 6, 7\}$ and set $k = 10$ (i.e., querying 10-NNs). The query patterns are randomly sampled from substrings that occur in the sequence collection. To better preserve the empirical distribution of the vector dataset, half of the query vectors are sampled directly from the original vector set, while the remaining half are generated by applying minor perturbations to randomly sampled vectors.

Remarks. OptQuery encounters out-of-memory (OOM) errors on the MTG, ArXiv, SwissProt, and CodeSearchNet datasets. Pgvector does not support constructing HNSW indexes for vectors with dimensionality larger than 2000 and is thus excluded from the Words dataset. PreFiltering is omitted in cases where its QPS is less than 10% of the lowest QPS achieved by the other methods. BM25Filtering exhibits a recall of only 0.01 on SwissProt dataset ($|p| = 4$ case) and is thereby omitted in the figure.

6.2 Query performance

Overall performance. Figure 8 reports the query performance of VECTORMATON and all baselines. For a result set R and a ground-truth set R^* , the answer quality is quantified by $Recall(R) = \frac{|R \cap R^*|}{|R^*|}$, as defined in Section 2.1. Overall, VECTORMATON consistently achieves one of the best recall-QPS trade-offs among all evaluated methods. When OptQuery is feasible, VECTORMATON delivers comparable performance while requiring substantially lower index overhead. Compared with the remaining baselines, VECTORMATON typically achieves both higher recall and higher query throughput. For example, on the CodeSearchNet dataset with pattern length $|p| = 4$, VECTORMATON achieves recalls close to 1, while simultaneously delivering up to two orders of magnitude higher QPS. Similar trends can be observed on all datasets, where VECTORMATON is consistently located on the Pareto frontier of the recall-QPS trade-off.

Discussion on baseline performances. We observe that the Hybrid method, which switches between PreFiltering and PostFiltering according to the query selectivity, consistently outperforms both individual methods. However, this improvement comes at the cost of higher indexing overhead. BM25Filtering introduces

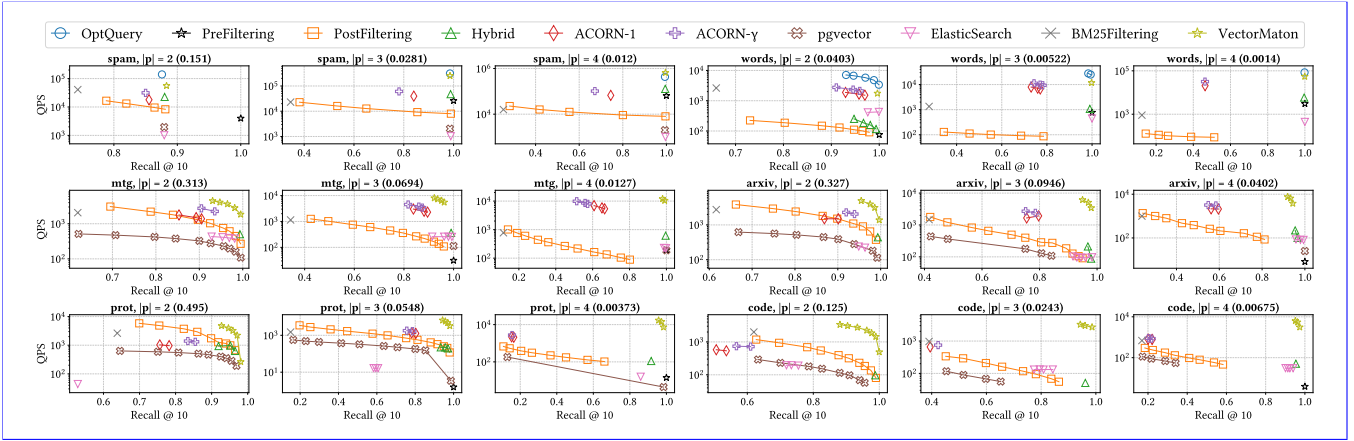


Figure 8: QPS vs. recall, where (x) in the title of each subplot denotes the average query selectivity is x.

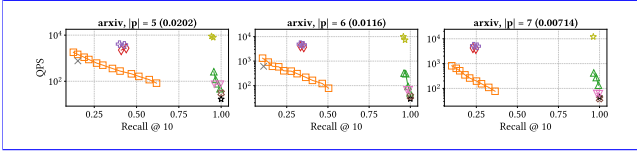


Figure 9: Long pattern queries on ArXiv dataset.

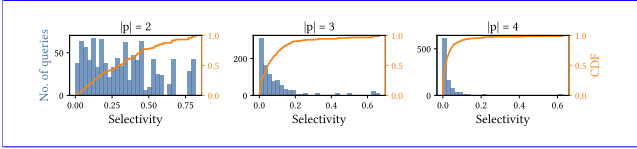


Figure 10: Query selectivity distribution on ArXiv dataset.

even higher indexing overhead while providing limited query-performance benefits. This suggests that a straightforward integration of inverted-index-based filtering with HNSW is insufficient for pattern-constrained vector search. Although the inverted index can effectively identify sequences sharing common n -grams with the query pattern, the HNSW graph associated with each n -gram posting list still contains many vectors whose sequences do not satisfy the full pattern constraint. Consequently, BM25Filtering suffers from a large number of false-positive candidates, leading to limitations similar to those of PostFiltering. ACORN generally achieves high query efficiency. However, because it performs filtering during graph traversal, a large fraction of vertices may be excluded under low-selectivity workloads, reducing the connectivity of the searchable graph and leading to a low recall. Although ACORN mitigates this issue by increasing vertex degrees and exploring additional 2-hop neighbors, these techniques cannot fundamentally overcome the sparsity induced by aggressive filtering. For example, on the SwissProt dataset with $|p| = 4$, the highest recall achieved by ACORN remains below 0.25. Pgvector adopts a hybrid strategy combining pre-filtering and post-filtering, and can occasionally achieve recall close to 1 under certain parameter settings. However, this comes at the expense of substantially lower query throughput. We also observe that adjusting `ef_search` for BM25Filtering yields no change in recall or throughput because the final candidate selection is largely dominated by BM25 ranking rather than ANN search quality, such that some ground-truth results are filtered out during the BM25 ranking process. This indicates that BM25 may provide a negative impact in this problem setting as it is more suitable for textual relevance rather than exact pattern containment.

Datasets	spam	words	mtg	arxiv	prot	code
Reduction (size)	93.9%	75.7%	80.6%	77.6%	72.9%	67.2%
Datasets	spam	words	mtg	arxiv	prot	code
Reduction (time)	99.4%	6.7%	58.4%	64.7%	22.2%	27.0%

Table 3: Reduction of index size and construction time using proposed index strategies.

Effects of varying $|p|$. To evaluate the impact of pattern selectivity, we vary the pattern length $|p| \in \{2, 3, 4, 5, 6, 7\}$ for each dataset. For short patterns $p \in [2, 4]$, the results are shown in Figure 8. As $|p|$ increases, the number of vectors satisfying the pattern constraint generally decreases, leading to lower selectivity. Under this setting, most baseline methods experience performance degradation, as they must expand the search space to compensate for the higher fraction of filtered-out results and maintain recall. In contrast, VECTORMATON and PreFiltering benefit from larger $|p|$. A longer pattern reduces the size of the filtered vector set V_p , thereby shrinking the effective search space of VECTORMATON and PreFiltering. As a result, both VECTORMATON and PreFiltering achieves improved query efficiency while maintaining high recall. For longer patterns with $|p| \in \{5, 6, 7\}$, we report the results on the ArXiv dataset in Figure 9; results for the remaining datasets are included in the appendix of our extended version [52]. We observe that the selectivity becomes extremely low for these pattern lengths. For example, when $|p| = 5$, the average selectivity is already below 1% on most datasets. Consequently, post-filtering-based approaches (e.g., PostFiltering and ACORN) suffer from severe recall degradation. Although PreFiltering achieves slightly higher query efficiency than in the short-pattern setting, it remains substantially slower than VECTORMATON, while VECTORMATON continues to maintain both high recall and superior query throughput under these workloads.

Pattern distribution analysis. We further analyze the distribution of query pattern selectivity in our workloads. Figure 10 presents the PDF and CDF of query selectivity on the ArXiv dataset, and we observe similar trends on the other datasets. Overall, the distribution tends to skew toward low-selectivity queries. As the pattern length increases, the distribution becomes increasingly concentrated in the low-selectivity region. In particular, the distribution for $|p| = 2$ is substantially less skewed than those for $|p| = 3$ and $|p| = 4$. This observation explains why post-filtering-based methods remain competitive when $|p| = 2$, but their recall deteriorates rapidly as $|p|$ increases and the query selectivity decreases.

6.3 Index construction and maintenance

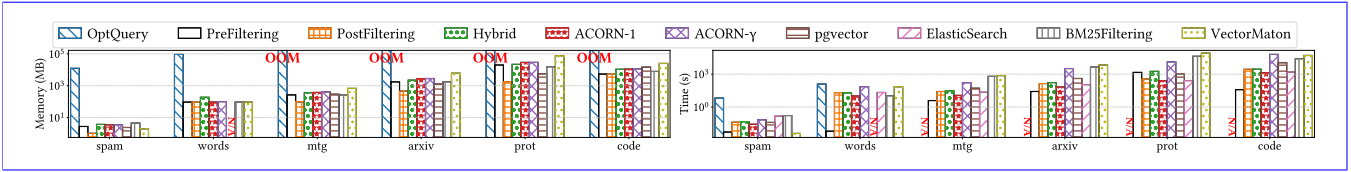


Figure 11: The peak memory consumption and time overhead of index construction.

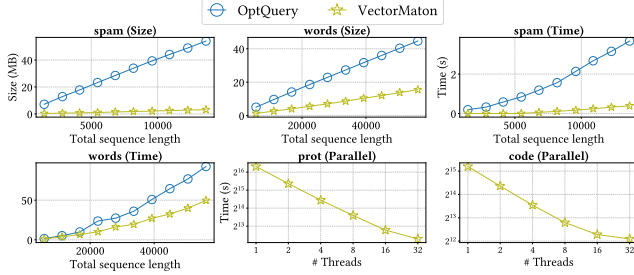


Figure 12: Index scalability test and parallelized index construction time.

Memory overhead. We evaluate the peak memory consumption during data preparation and index construction in Figure 11. We note that ElasticSearch runs within a JVM configured with a fixed pre-allocated heap size, making its actual memory usage difficult to measure fairly; therefore, we exclude it from this comparison. PostFiltering and pgvector exhibit the lowest memory consumption because they only build a single HNSW index without any adjustments. However, this simplicity comes at the cost of low recall or efficiency. For example, on SwissProt dataset with $|p| = 4$, pgvector can only answer less than 10 queries per second, making them impractical for actual use. To accelerate pattern filtering, we integrate PreFiltering, Hybrid, and ACORN with our ESAM implementation. As a result, these methods require additional memory beyond their original designs. VECTORMATON incurs higher memory consumption than most baseline methods due to the need to build multiple HNSW graphs based on the sequences, but this overhead is compensated by substantially superior query performance. On small-to-moderate datasets, VECTORMATON requires an affordable memory of less than 10GB to build the index.

Time overhead. Figure 11 reports the index construction time of all methods. PreFiltering, PostFiltering, Hybrid, ACORN-1, pgvector, and ElasticSearch exhibit relatively low construction overhead because they either require negligible indexing structures (PreFiltering) or maintain only a single HNSW graph (the remaining methods). ACORN-γ incurs substantially higher construction time than ACORN-1 due to the additional graph expansion required to increase vertex connectivity and incorporate multi-hop neighborhood information. In contrast, OptQuery, BM25Filtering, and VECTORMATON incur higher construction overhead because they maintain multiple HNSW graphs to support pattern-constrained search. Among them, OptQuery and VECTORMATON effectively trade index construction time for significantly improved query performance. BM25Filtering also incurs considerable construction overhead because it builds HNSW indexes for the posting lists associated with individual 2-grams. However, unlike OptQuery and VECTORMATON, this additional indexing cost does not translate into corresponding query-performance gains. Overall, the results indicate that VECTORMATON achieves a favorable trade-off between index construction overhead and query efficiency, whereas BM25Filtering introduces substantial indexing costs while providing limited benefits during query processing. We also remark that the construction overhead

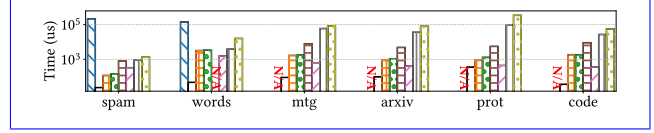


Figure 13: The average insertion time of 80% base dataset and 20% insertion data (ACORN-1 and ACORN-γ are excluded).

of VECTORMATON can be substantially reduced through parallel index construction (see Section 6.4 “parallel index construction”).

Index scalability test. We evaluate the scalability of index size and construction time on Spam and Words datasets for OptQuery and VECTORMATON methods. For each dataset, we construct the index over progressively larger subsets $\{10\%, 20\%, \dots, 100\%\}$ of the data and plot the resulting size-length and time-length curves. Figure 12 (spam, words) illustrates the results. Although the theoretical worst-case space complexity is $O(m^2)$ for OptQuery and $O(m^{1.5})$ for VECTORMATON, the empirical space costs exhibit near-linear growth in practice for both methods. This behavior arises because real sequences typically contain repeated substrings, resulting in fewer distinct patterns to index than the worst-case bound. In addition, we observe the growth rate of VECTORMATON is significantly lower than that of OptQuery. As the dataset size increases, the gap between the two methods becomes larger. These results demonstrate that VECTORMATON scales gracefully with increasing data size and can effectively handle large-scale sequence collections.

Index insertion. We evaluate the index insertion overhead of all methods. Specifically, for each dataset, we first construct the index on 80% of the data and subsequently insert the remaining 20%. ACORN-1 and ACORN-γ do not support dynamic updates and are therefore excluded from this experiment. Since the memory overhead during insertion is similar to that during index construction, we focus on insertion time in this evaluation. Figure 13 reports the average insertion time of all methods. Overall, we observe that the insertion overheads of most methods are proportional to their corresponding index construction overheads, as most methods inherently construct their index via an incremental index maintenance. For VECTORMATON, the proposed insertion scheme is the amortized cost of the index construction time, and does not introduce additional asymptotic complexity. Consequently, its insertion overhead remains moderate. Among the baselines, PreFiltering incurs the lowest insertion cost because it does not maintain any HNSW graph. ElasticSearch also exhibits relatively low insertion overhead due to its segment-based architecture. Newly inserted vectors are only added to the HNSW graph of a newly created segment rather than a global HNSW graph over the entire dataset. As a result, insertion cost depends primarily on the segment size rather than the total dataset size. However, query processing must search multiple segment-level HNSW graphs and merge their results, while periodic segment merges incur additional maintenance overhead.

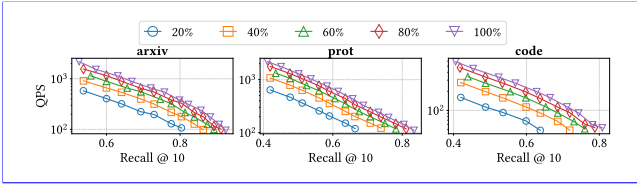


Figure 14: QPS-recall curves of PostFiltering with different candidate set size ratio.

6.4 Case studies

Parallelized index construction. We evaluate the parallel index construction on two large-scale datasets, SwissProt and CodeSearch-Net. For each dataset, we construct the index using 1, 2, 4, 8, 16, and 32 threads, respectively. Figure 12 (prot, code) reports the results. The results show that VECTORMATON scales effectively in a parallel setting, achieving linear speedup when the number of threads does not exceed 16. Beyond this point, the speedup decreases. This is because our parallelization strategy distributes the construction of different state indexes across threads, while the construction of each individual index remains sequential. As the number of threads increases, the overall runtime becomes dominated by a few large state indexes whose construction cannot be further parallelized, leading to reduced scalability gains. Nevertheless, with 16 threads, the index construction time is reduced to less than two hours on both datasets, which is efficient for large-scale datasets.

Ablation study of index strategies. Recall that VECTORMATON incorporates two design strategies: index reuse across states and selective index construction (skip-build). We conduct an ablation study to evaluate their effects with the results in Table 3, showing that they effectively reduce both index size and time, with an up to 93.9% reduction in size and 99.4% reduction in time. The reason is that index reuse strategy reduces redundancy of overlapping vector subsets across related states, while the skip-build strategy avoids unnecessary HNSW construction for small ID sets.

Effects of parameter T . In the skip-build strategy, we introduce a threshold parameter T to determine whether an HNSW index should be constructed for a state. We evaluate the impact of varying T . For space limit, detailed results and analysis are deferred to the appendix of our extended version [52].

Parameter tuning of PostFiltering. Recall that in PostFiltering (Algorithm 1), the HNSW search is executed with parameter ef_search , and the top- ef_search candidates are retrieved before applying pattern filtering. Thus, the candidate set size is determined by ef_search . The intuition behind this design is that a larger candidate set increases the likelihood of retaining at least k valid results after filtering, thereby improving recall. To validate this intuition, we vary the candidate set size by returning only the top- k_s candidates from the initial HNSW search, where $k_s \in \{20\%, 40\%, 60\%, 80\%, 100\%\} \cdot ef_search$. Queries are generated with mixed pattern lengths $|p| \in \{2, 3, 4\}$. Figure 14 reports the results. We observe that reducing k_s consistently degrades the query performance of PostFiltering. Therefore, our default configuration $k_s = ef_search$ provides the best overall performance.

6.5 Applicable long-sequence scenario

Although our main evaluation covers real-world datasets with average sequence lengths ranging from fewer than 10 symbols to more than 100 symbols, some emerging applications involve longer sequences. One representative example is retrieval-augmented generation (RAG) systems [31], where documents are partitioned into

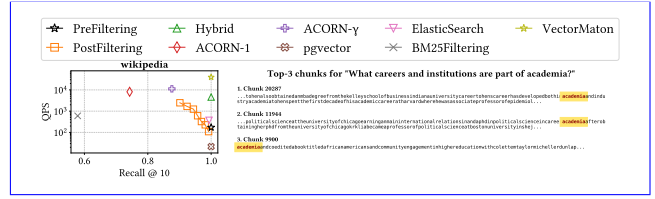


Figure 15: Case study of Wikipedia.

chunks and represented as vectors for semantic retrieval. In practice, users often combine semantic search with keyword or phrase constraints (e.g., requiring a retrieved chunk to contain a specific entity or term). Such queries naturally correspond to the ANNS with pattern constraints problem studied in this paper.

To evaluate query performance in this setting, we conduct an additional case study using Wikipedia articles [51]. Specifically, we select 10,000 articles and partition them into text chunks. Each chunk is embedded into a vector using a text embedding model, while the original chunk text serves as the associated sequence. The resulting dataset contains approximately 20 million characters in total, with many chunks exceeding 1,000 characters, making the sequences substantially longer than those in our main benchmark datasets. We further construct 75 realistic query workloads consisting of an embedded prompt and a keyword constraint generated by ChatGPT [42]. For example, the prompt “What careers and institutions are part of academia?” is paired with the keyword constraint “academia”. Figure 15 reports the results. We observe that VECTORMATON achieves the highest query throughput while achieving recall 1.0, demonstrating its effectiveness on long-sequence workloads representative of practical RAG applications. The top-ranked result for the example query discusses the educational background, academic positions, and affiliated institutions of a researcher, while explicitly containing the keyword “academia”. This demonstrates that VECTORMATON successfully retrieves semantically relevant content that also satisfies the specified keyword constraint.

7 RELATED WORKS

ANNS approaches. ANNS methods can be generally classified into four categories: (1) tree-based methods [9, 22, 41]; (2) hashing-based methods [3, 14, 50, 56]; (3) quantization-based methods [4, 6, 17, 18]; and (4) graph-based methods [16, 36, 48]. Tree-based methods partition the data space hierarchically and organize points into recursive structures, including KD-tree [9], ball tree [41], and ring-cover tree [22]. Hashing-based methods include locality-sensitive hashing (LSH) [3, 50, 56] and SimHash [14]. Quantization-based methods [4, 6, 17, 18] encode high-dimensional vectors into compact representations using learned codebooks and approximate distances in the compressed space. Graph-based methods construct a proximity graph (e.g., HNSW [36], NSG [16]) over the dataset and perform query processing via greedy graph traversal.

Filtered ANNS approaches. Existing filtered ANNS approaches mainly focus on attribute-based filtering [20, 32, 34, 38] and range-based filtering [25, 45, 53, 54, 57]. Attribute-based filtering includes selecting vectors whose categorical attribute equals a queried value [20], or whose attribute set contains the queried set [34]. Range-based filtering considers numerical constraints associated with each vector [25, 45, 53, 54, 57]. The objective is to answer ANN queries while restricting results to vectors whose associated numeric value falls within a query interval. Recently, a few studies (e.g., ACORN [44]) have investigated general filtered ANNS, with some [33, 55] appeared contemporaneously with this paper.

8 CONCLUSION

In this paper, for the first time, we introduce the problem of ANNS with pattern constraints. To address this problem, we propose VECTORMATON, an index that jointly supports similarity search and pattern filter. Experimental results show that VECTORMATON achieves a favorable trade-off among query latency, recall, and index space consumption compared with baseline methods. Moreover, the index size grows empirically linearly with respect to the dataset size.

REFERENCES

- [1] 2025. pgvector. <https://github.com/pgvector/pgvector>.
- [2] 2025. Vector search in Elasticsearch. <https://www.elastic.co/docs/solutions/search/vector>.
- [3] Alexandr Andoni and Piotr Indyk. 2008. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Commun. ACM* 51, 1 (Jan. 2008), 117–122. <https://doi.org/10.1145/1327452.1327494>
- [4] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache locality is not enough: high-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.* 9, 4 (Dec. 2015), 288–299. <https://doi.org/10.14778/2856318.2856324>
- [5] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2025. Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art. *Proc. ACM Manag. Data* 3, 1, Article 43 (Feb. 2025), 31 pages. <https://doi.org/10.1145/3709693>
- [6] Artem Babenko and Victor Lempitsky. 2015. Tree quantization for large-scale similarity search and classification. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 4240–4248. <https://doi.org/10.1109/CVPR.2015.7299052>
- [7] Timothy L Bailey, Mikael Boden, Fabian A Buske, Martin Frith, Charles E Grant, Luca Clementi, Jingyuan Ren, Wilfred W Li, and William S Noble. 2009. MEME SUITE: tools for motif discovery and searching. *Nucleic Acids Res* 37, Web Server issue (May 2009), W202–8.
- [8] Yoshua Bengio, Aaron Courville, and Pascal Vincent. 2013. Representation Learning: A Review and New Perspectives. *IEEE Trans. Pattern Anal. Mach. Intell.* 35, 8 (Aug. 2013), 1798–1828. <https://doi.org/10.1109/TPAMI.2013.50>
- [9] Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Commun. ACM* 18, 9 (Sept. 1975), 509–517. <https://doi.org/10.1145/361002.361007>
- [10] A. Blumer, J. Blumer, A. Ehrenfeucht, D. Haussler, and R. McConnell. 1984. Building the minimal DFA for the set of all subwords of a word on-line in linear time. In *Automata, Languages and Programming*, Jan Paredaens (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 109–118.
- [11] A. Blumer, J. Blumer, D. Haussler, A. Ehrenfeucht, M.T. Chen, and J. Seiferas. 1985. The smallest automation recognizing the subwords of a text. *Theoretical Computer Science* 40 (1985), 31–55. [https://doi.org/10.1016/0304-3975\(85\)90157-4](https://doi.org/10.1016/0304-3975(85)90157-4)
- [12] A. Blumer, J. Blumer, D. Haussler, R. McConnell, and A. Ehrenfeucht. 1987. Complete inverted files for efficient text retrieval and analysis. *J. ACM* 34, 3 (July 1987), 578–595. <https://doi.org/10.1145/28869.28873>
- [13] Nadav Brandes, Dan Ofer, Yam Peleg, Nadav Rappoport, and Michal Linial. 2022. ProteinBERT: a universal deep-learning model of protein sequence and function. *Bioinformatics* 38, 8 (02 2022), 2102–2110. <https://doi.org/10.1093/bioinformatics/btac020> arXiv:https://academic.oup.com/bioinformatics/article-pdf/38/8/2102/49009610/btac020.pdf
- [14] Moses S. Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *Proceedings of the Thirty-Fourth Annual ACM Symposium on Theory of Computing* (Montreal, Quebec, Canada) (STOC '02). Association for Computing Machinery, New York, NY, USA, 380–388. <https://doi.org/10.1145/509907.509965>
- [15] Efarall. 2026. Word Embeddings. https://huggingface.co/datasets/efarall/word_embeddings. Hugging Face dataset.
- [16] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.* 12, 5 (Jan. 2019), 461–474. <https://doi.org/10.14778/3303753.3303754>
- [17] Jianyang Gao, Yutong Gou, Yuxuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. 2025. Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3, Article 202 (June 2025), 26 pages. <https://doi.org/10.1145/3725413>
- [18] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 3, Article 167 (May 2024), 27 pages. <https://doi.org/10.1145/3654970>
- [19] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB '99)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 518–529.
- [20] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023* (Austin, TX, USA) (WWW '23). Association for Computing Machinery, New York, NY, USA, 3406–3416. <https://doi.org/10.1145/3543507.3583552>
- [21] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable Feature Learning for Networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) (KDD '16). Association for Computing Machinery, New York, NY, USA, 855–864. <https://doi.org/10.1145/2939672.2939754>
- [22] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing* (Dallas, Texas, USA) (STOC '98). Association for Computing Machinery, New York, NY, USA, 604–613. <https://doi.org/10.1145/276698.276876>
- [23] Ird. 2026. Code search net. <https://huggingface.co/datasets/irds/codesearchnet>. Hugging Face dataset.
- [24] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (Jan. 2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [25] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibow Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proc. ACM Manag. Data* 3, 3, Article 148 (June 2025), 26 pages. <https://doi.org/10.1145/3725399>
- [26] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>
- [27] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. 2021. Highly accurate protein structure prediction with AlphaFold. *Nature* 596, 7873 (01 Aug 2021), 583–589. <https://doi.org/10.1038/s41586-021-03819-2>
- [28] Khairi. 2026. Uniprot swissprot. <https://huggingface.co/datasets/khairi/uniprot-swissprot>. Hugging Face dataset.
- [29] Donald E. Knuth. 1998. *The art of computer programming, volume 3: (2nd ed.) sorting and searching*. Addison Wesley Longman Publishing Co., Inc., USA.
- [30] Quoc Le and Tomas Mikolov. 2014. Distributed Representations of Sentences and Documents. In *Proceedings of the 31st International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Eric P. Xing and Tony Jebara (Eds.), Vol. 32. PMLR, Beijing, China, 1188–1196. <https://proceedings.mlr.press/v32/le14.html>
- [31] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [32] Mocheng Li, Xiao Yan, Baotong Lu, Yue Zhang, James Cheng, and Chenhao Ma. 2025. Attribute Filtering in Approximate Nearest Neighbor Search: An In-depth Experimental Study. *Proc. ACM Manag. Data* 3, 6, Article 298 (Dec. 2025), 26 pages. <https://doi.org/10.1145/3769763>
- [33] Zhiyue Li, Guangyan Zhang, Ruochun Jin, Bojun Li, Xiaoguang Ren, and Wenjing Yang. 2026. PathSeer: Adaptive Neighbor Handling for Efficient Filtered ANN Search. *Proc. ACM Manag. Data* 4, 3, Article 221 (May 2026), 27 pages. <https://doi.org/10.1145/3802098>
- [34] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *Proc. VLDB Endow.* 18, 4 (Dec. 2024), 1118–1130. <https://doi.org/10.14778/3717755.3717770>
- [35] Shangqi Lu and Yufei Tao. 2025. Proximity Graphs for Similarity Search: Fast Construction, Lower Bounds, and Euclidean Separation. *Proc. ACM Manag. Data* 3, 5, Article 280 (Nov. 2025), 25 pages. <https://doi.org/10.1145/3767716>
- [36] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (April 2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [37] Malteos. 2026. Aspect Paper Embeddings. <https://huggingface.co/datasets/Qdrant/arxiv-titles-instructorxl-embeddings>. Hugging Face dataset.
- [38] Yusuke Matsui, Ryota Hinami, and Shin'ichi Satoh. 2018. Reconfigurable Inverted Index. In *Proceedings of the 26th ACM International Conference on Multimedia* (Seoul, Republic of Korea) (MM '18). Association for Computing Machinery, New York, NY, USA, 1715–1723. <https://doi.org/10.1145/3240508.3240630>

- [39] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. *arXiv:1301.3781 [cs.CL]* <https://arxiv.org/abs/1301.3781>
- [40] Mehryar Mohri, Pedro Moreno, and Eugene Weinstein. 2009. General suffix automaton construction algorithm and space bounds. *Theoretical Computer Science* 410, 37 (2009), 3553–3562. <https://doi.org/10.1016/j.tcs.2009.03.034> Implementation and Application of Automata (CIAA 2007).
- [41] Stephen M. Omohundro. 2009. Five Balltree Construction Algorithms. <https://api.semanticscholar.org/CorpusID:61067117>
- [42] OpenAI. 2026. ChatGPT. <https://chatgpt.com>.
- [43] Malte Ostendorff, Till Blume, Terry Ruas, Bela Gipp, and Georg Rehm. 2022. Specialized document embeddings for aspect-based similarity of research papers. In *Proceedings of the 22nd ACM/IEEE Joint Conference on Digital Libraries (Cologne, Germany) (JCDL '22)*. Association for Computing Machinery, New York, NY, USA, Article 7, 12 pages. <https://doi.org/10.1145/3529372.3530912>
- [44] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 3, Article 120 (May 2024), 27 pages. <https://doi.org/10.1145/3654923>
- [45] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 10 (June 2025), 3256–3268. <https://doi.org/10.14778/3748191.3748193>
- [46] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Found. Trends Inf. Retr.* 3, 4 (April 2009), 333–389. <https://doi.org/10.1561/15000000019>
- [47] SpamAssassin. 2026. Spam emails. <https://spamassassin.apache.org/old/publiccorpus/>. Hugging Face dataset.
- [48] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. *DiskANN: fast accurate billion-point nearest neighbor search on a single node*. Curran Associates Inc., Red Hook, NY, USA.
- [49] TrevorJS. 2026. Mtg Scryfall Cropped Art Embeddings. <https://huggingface.co/datasets/TrevorJS/mtg-scrryfall-cropped-art-embeddings-siglip-so400m-patch14-384>. Hugging Face dataset.
- [50] Jiuqi Wei, Botao Peng, Xiaodong Lee, and Themis Palpanas. 2024. DET-LSH: A Locality-Sensitive Hashing Scheme with Dynamic Encoding Tree for Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 17, 9 (May 2024), 2241–2254. <https://doi.org/10.14778/3665844.3665854>
- [51] Wikimedia. 2026. Dataset Card for Wikimedia Wikipedia. <https://huggingface.co/datasets/wikimedia/wikipedia>. Hugging Face dataset.
- [52] Haoxuan Xie and Siqiang Luo. 2026. VectorMaton: Efficient Vector Search with Pattern Constraints via an Enhanced Suffix Automaton. https://github.com/ForwardStar/VectorMaton/blob/main/technical_report.pdf.
- [53] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6, Article 239 (Dec. 2024), 26 pages. <https://doi.org/10.1145/3698814>
- [54] Fangyuan Zhang, Mengxu Jiang, Guanhao Hou, Jieming Shi, Hua Fan, Wenchao Zhou, Feifei Li, and Sibao Wang. 2025. Efficient Dynamic Indexing for Range Filtered Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3, Article 152 (June 2025), 26 pages. <https://doi.org/10.1145/3725401>
- [55] Xiang Zhang, Chao Zhang, Ju Fan, Guoliang Li, and Xiaoyong Du. 2026. VecBench: A Controllable Benchmark for Filtered Vector Search: [Experiments & Analysis]. *Proc. ACM Manag. Data* 4, 3, Article 248 (May 2026), 27 pages. <https://doi.org/10.1145/3802125>
- [56] Bolong Zheng, Xi Zhao, Lianggui Weng, Nguyen Quoc Viet Hung, Hang Liu, and Christian S. Jensen. 2020. PM-LSH: A fast and accurate LSH framework for high-dimensional approximate NN search. *Proc. VLDB Endow.* 13, 5 (Jan. 2020), 643–655. <https://doi.org/10.14778/3377369.3377374>
- [57] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 1, Article 69 (March 2024), 26 pages. <https://doi.org/10.1145/3639324>

A PSEUDO CODES OF OPTQUERY

Algorithm 3: OptQuery

```

1 Function Build( $n, V, S$ ):
2    $H \leftarrow$  a hashmap that maps patterns to HNSW graphs;
3   for  $i \leftarrow 1, 2, \dots, n$  do
4     for  $j \leftarrow 1, 2, \dots, |s_i|$  do
5       for  $k \leftarrow j, j+1, \dots, |s_i|$  do
6         Extract substring  $s_i[j..k]$  as a pattern  $p$ ;
7         Insert  $v_i$  to the graph of  $H(p)$ ;

8 Function Query( $v_q, p, k, ef\_search$ ):
9   if  $H(p)$  does not exist then return  $\emptyset$ ;
10  Search in the HNSW graph  $H(p)$  with parameters
     $v_q, k, ef\_search$  and return  $k$  results;

```

B PROOF OF LEMMA 1

Consider a maximal pattern whose position list contains a pair (id, pos) . Such a pattern must be a substring of the form $s_{id}[i : pos]$, representing the substring from position i to pos in sequence s_{id} . For $0 \leq i < pos$, suppose that $s_{id}[i : pos]$ and $s_{id}[i+1 : pos]$ are not equivalent. Then, by the definition of equivalence classes, $s_{id}[i+1 : pos]$ must occur in some other pair (id', pos') , while $s_{id}[i : pos]$ does not. In other words, for x maximal patterns whose position lists contain (id, pos) , there must exist at least $x-1$ occurrences of these patterns associated with other (id', pos') pairs.

Now, to count the total number of distinct maximal patterns, we sum over all (id, pos) pairs. For each pair, the contribution to the total number of distinct maximal patterns is at most $x_{id,pos} - (x_{id,pos} - 1) = 1$. Therefore, summing over all such pairs, the total number of distinct maximal patterns is at most $\sum_{(id,pos)} 1 = m$, where m is the total length of all sequences.

C PROOF OF LEMMA 2

Consider a sequence s of length $|s|$. It has at most $\frac{|s|(|s|+1)}{2}$ distinct substrings, and thus can contribute its sequence ID to at most that many maximal patterns. By Lemma 1, the total number of states in VECTORMATON is bounded by $O(m)$; denote this bound by cm for some constant c . Therefore, the ID i of sequence s_i can appear in at most: $\min\left\{\frac{|s_i|(|s_i|-1)}{2}, cm\right\}$ states. Let S denote the sequence collection. The total size of all associated ID sets is thus bounded by $\sum_{i=1}^n \min\left\{\frac{|s_i|(|s_i|-1)}{2}, cm\right\}$. Next, we denote $t_i = \min\left\{\frac{|s_i|^2}{2}, cm\right\} \geq \min\left\{\frac{|s_i|(|s_i|-1)}{2}, cm\right\}$ and consider two cases.

- If $\frac{|s_i|^2}{2} \leq cm$, then $|s_i| \leq \sqrt{2cm}$ and $t_i = \frac{|s_i|^2}{2} \leq |s_i| \cdot \frac{\sqrt{2cm}}{2}$.
- If $\frac{|s_i|^2}{2} > cm$, then $\sqrt{m} < \frac{|s_i|\sqrt{2c}}{2}$ and $t_i = cm < |s_i| \cdot \frac{\sqrt{2cm}}{2}$.

Therefore, the total size of all associated ID sets is bounded by:

$$\sum_{i=1}^n t_i \leq \sum_{i=1}^n |s_i| \frac{\sqrt{2cm}}{2} = \frac{\sqrt{2c}}{2} m^{1.5} = O(m^{1.5})$$

D PROOF OF LEMMA 3

Let $poslist(p_A) = \{(id_j, pos_j)\}_{j=1}^k$. Since $p_B = p_A \cdot c$, an occurrence of p_A at position (id_j, pos_j) can be extended by c if and

only if $pos_j < |s_{id_j}| - 1$ and the next character is c . Therefore, $poslist(p_B) = \{(id_j, pos_j + 1) \mid 1 \leq j \leq k, s_{id_j}[pos_j + 1] = c\}$. For any other pattern $p'_A \in A$, by definition of equivalence classes we have $poslist(p'_A) = poslist(p_A)$. Hence, extending p'_A by the same character c yields the same set of occurrences and thus the same position list as p_B . Consequently, $p'_A \cdot c$ belongs to the same equivalence class B , which establishes a well-defined transition from state A to state B labeled by c .

E PROOF OF LEMMA 4

In index reuse strategy, we construct I_j only over the difference set $V_j \setminus I_k$. Therefore, it directly draws the result that $I_j \cup I_k = V_j$ and $I_j \cap I_k = \emptyset$.

F PROOF OF LEMMA 5

Let A be a suffix state with position list $poslist(A) = (x_k, y_k)$. If there is no outgoing transition from A labeled c , then for every $(x_k, y_k) \in poslist(A)$, either $y_k + 1$ exceeds the length of s_{x_k} or $s_{x_k}[y_k + 1] \neq c$. Hence, none of the existing occurrences of patterns in A can be extended by c . Since we are currently processing position (i, j) and the next symbol is $c = s_i[j + 1]$, appending c introduces a new occurrence ending at $(i, j + 1)$. Therefore, extending A by c yields a new equivalence class with position list $\{(i, j + 1)\}$.

G SUFFIX LINK AND ANALYSIS

G.1 Correctness

In this section, we will prove the correctness of our automaton construction algorithm in an on-line fashion (i.e., the correctness of the incremental process). We first present the definition of the *suffix link* [10]. While it is originally introduced for the single-sequence setting, the concept naturally extends to our multi-sequence scenario.

DEFINITION 6 (SUFFIX LINK). *Given a state A whose maximal pattern is s , the suffix link $link(A)$ points to the state whose maximal pattern is the longest proper suffix of s that is not equivalent to s .*

We next show that all suffix states can be organized into a chain of strictly decreasing maximal pattern lengths via suffix links.

LEMMA 6. *Consider sequence s_i currently being processed, and suppose the algorithm has processed position j . Let A be the state corresponding to pattern $s_i[0..j]$. Then the set of all suffix states is precisely $\{A, link(A), link(link(A)), \dots, 0\}$, where 0 denotes the initial state.*

PROOF. By definition of suffix states, a suffix state is a state whose position list is empty or contains (i, j) . Hence, every suffix state represents at least a pattern that is a suffix of $s_i[0..j]$. Moreover, the maximal pattern in a suffix state must be also a suffix of $s_i[0..j]$, which is not hard to see.

Let A be the state corresponding to $s_i[0..j]$. Then A is a suffix state with maximal pattern $s_i[0..j]$. By construction of suffix links, $link(A)$ points to the state representing the longest proper suffix of the maximal pattern of A that belongs to a different equivalence class. Repeatedly following suffix links therefore enumerates states whose maximal patterns are exactly the suffixes of $s_i[0..j]$ in strictly decreasing order of length, until reaching the initial state 0 , which

represents the empty pattern. Each of these states contains (i, j) in its position list (or is the initial state), and is thus a suffix state.

Conversely, let C be any suffix state. Then C represents a pattern that is a suffix of $s_i[0..j]$. By the defining property of suffix links in the (generalized) suffix automaton, every suffix of $s_i[0..j]$ is represented by a state on the suffix-link chain starting from A . Therefore, C must belong to the set $\{A, \text{link}(A), \text{link}(\text{link}(A)), \dots, 0\}$. $\square$

Next, we show that the suffix-link chain of suffix states can be partitioned into at most two contiguous segments: (1) states without an outgoing transition labeled $c = s_i[j+1]$, and (2) states with such a transition.

LEMMA 7. *Let A be the state corresponding to pattern $s_i[0..j]$, and let $c = s_i[j+1]$ be the next symbol to be processed. Then the suffix-link chain $\{A, \text{link}(A), \text{link}^2(A), \dots, 0\}$ can be divided into at most two contiguous segments such that:*

- *In the first segment (if it exists), no state has an outgoing transition labeled c .*
- *In the second segment (if it exists), every state has an outgoing transition labeled c .*

PROOF. Let A be the state corresponding to the pattern $s_i[0..j]$, and consider the suffix-link chain $\{A, \text{link}(A), \text{link}^2(A), \dots, 0\}$.

If A has an outgoing transition labeled c , then the extended pattern $s_i[0..j+1]$ is already represented in the successor state. By the construction of suffix links, all states reachable via link from A correspond to shorter suffixes of $s_i[0..j]$. Since $s_i[0..j+1]$ contains these suffixes, each of these states must also have an outgoing transition labeled c .

Otherwise, if A has no transition labeled c , we follow the suffix links until we reach the first state B that does have a c -transition (if any). All states before B lack such a transition, while all states from B onward have it. If no state along the chain has a c -transition, then the entire chain forms a single segment without a c -transition. $\square$

For the first segment, by Lemma 5, we create a new state and connect all states in that segment to it via transitions labeled c . For the second segment, we only need to examine the first state in the segment.

LEMMA 8. *Let B be the first state in the second segment of the suffix-link chain, and let C be the successor of B via the transition labeled c . Let $s(B)$ and $s(C)$ denote the maximal patterns of states B and C , respectively. If $|s(C)| = |s(B)| + 1$, then extending by c does not introduce any structural change in the second segment.*

PROOF. We first establish the following claim.

Claim. Let X be a state with maximal pattern $s(X)$. Then every pattern y in the equivalence class of X is a suffix of $s(X)$.

Proof of claim. Let (i, j) be any occurrence in the position list of $s(X)$. Since all patterns in state X share the same position list, the pattern $y \in X$ must also end at position j in sequence s_i . Hence, $s(X) = s_i[j - |s(X)| + 1..j]$ and $y = s_i[j - |y| + 1..j]$, which shows that y is a suffix of $s(X)$.

Now consider the main statement. Since $|s(C)| = |s(B)| + 1$, the maximal pattern of C is exactly the extension of $s(B)$ by c , i.e., $s(C)$ is a suffix of $s_i[0..j+1]$. By the claim, all patterns represented in state C are suffixes of $s(C)$, and they are thereby suffixes of $s_i[0..j+1]$. Thus, when the currently processed sequence is extended by c at position $(i, j+1)$, all patterns in C remain consistent with this

extension and belong to the same equivalence class. Consequently, no refinement of state C is required, and no structural change occurs in the second segment. $\square$

We now consider the remaining case where $|s(C)| > |s(B)| + 1$. This implies that state C represents patterns strictly longer than the direct extension of $s(B)$ by c . In this situation, the extension by c introduces a refinement of the equivalence relation represented by C , and the state must be split.

LEMMA 9. *Let B be the first state in the second segment of the suffix-link chain, and let C be the successor of B via the transition labeled c . Let $s(B)$ and $s(C)$ denote the maximal patterns of states B and C , respectively. If $|s(C)| > |s(B)| + 1$, then for every state B' in the second segment whose c -transition points to C , the equivalence class obtained by extending B' with c is no longer correctly represented by C .*

PROOF. By our previous claim, $s(B) + c$ belongs to C and should be a suffix of $s(C)$. Since $s(B)$ is a suffix of $s_i[0..j]$, either $s(C)$ is a suffix of $s_i[0..j+1]$ or $s_i[0..j+1]$ is a substring of $s(C)$. For the latter case, the conclusion is trivial as $s(C)$ does not occur at $(i, j+1)$, violating the equivalence class of $s(B) + c$. For the former case, this indicates there exists some other state D which extends a sequence $x \in D$ to $s(C)$ via transition c , and x is thereby a suffix of $s_i[0..j]$. Therefore, D should be within the suffix-link chain and x is longer than $s(B)$. However, since B is the first state in the second segment, all previous states with longer suffixes of $s_i[0..j]$ should have no outgoing transition labeled c , which leads to a contradiction. $\square$

After splitting state C into C and its clone C' , we maintain the invariant that $\text{poslist}(C') = \text{poslist}(C) \cup \{(i, j+1)\}$, while C retains its original position list. We now determine precisely which transitions must be redirected to C' .

Let X be a suffix state in the second segment whose c -transition previously pointed to C . Then its maximal pattern occurs at position (i, j) . Extending this pattern by $c = s_i[j+1]$ therefore produces an occurrence ending at $(i, j+1)$. Consequently, the extended pattern has position list equal to $\text{poslist}(C')$, and must be represented by state C' . Hence the outgoing c -transition of X must be redirected to C' .

Conversely, let Y be any state that is not a suffix state whose c -transition points to C . Extending its maximal pattern by c does not yield an occurrence ending at $(i, j+1)$, and therefore its extended pattern does not include $(i, j+1)$ in its position list. The corresponding equivalence class thus remains $\text{poslist}(C)$, and the transition of Y continues to point to C . No redirection is required.

Therefore, exactly the suffix states in the second segment require their c -transitions to be redirected from C to C' . Combining all above analysis, we conclude that after each extension step, the automaton maintains correct to represent all equivalence classes.

G.2 Maintenance of suffix links

As discussed above, processing each symbol creates at most two new states. We now describe how suffix links are maintained. For the state created by cloning, its suffix link is set as the same suffix link value of original state being cloned. For the newly created state Q that is introduced for the first segment (i.e., states without an outgoing transition labeled c), its suffix link depends on whether the second segment exists:

- If the second segment does not exist (i.e., no suffix state has an outgoing transition labeled c), then $\text{link}(Q)$ is set to the initial state 0.
- Otherwise, let B be the first state in the second segment, and let C be its successor via the transition labeled c after any necessary cloning has been performed. Then we set $\text{link}(Q) = C$.

This maintains the invariant that the suffix link of a state points to the state representing the longest proper suffix of its maximal pattern.

LEMMA 10. *The suffix link $\text{link}(Q)$ constructed as above is correct.*

PROOF. Let $s(Q) = s_i[0..j + 1]$ denote the maximal pattern of the newly created state Q .

Case 1: The second segment does not exist. This means that no suffix state of $s_i[0..j]$ has an outgoing transition labeled c . Hence, for every proper suffix x of $s_i[0..j]$, the extension $x + c$ does not occur in the previously processed prefix. Consequently, among all proper suffixes of $s(Q)$, none except the empty pattern corresponds to an existing state in a different equivalence class. Therefore, the longest proper suffix of $s(Q)$ that belongs to a different equivalence class is the empty pattern, represented by the initial state 0. Thus $\text{link}(Q) = 0$ is correct.

Case 2: The second segment exists. Let B be the first state in the second segment, and let C be the successor of B via transition labeled c after any necessary structural updates. Since B lies on the suffix-link chain of $s_i[0..j]$, $s(B)$ is a suffix of $s_i[0..j]$, and hence $s(B) + c$ is a proper suffix of $s(Q)$. Moreover, all suffix states preceding B (which correspond to strictly longer suffixes of $s_i[0..j]$) do not have a transition labeled c . Therefore, no longer proper suffix of $s(Q)$ corresponds to an existing state in a different equivalence class. Thus $s(B) + c$ is the longest proper suffix of $s(Q)$ that belongs to a distinct equivalence class, and it is represented by state C . Hence, setting $\text{link}(Q) = C$ satisfies the suffix-link invariant. $\square$

H PROOF OF THEOREM 1

By Lemma 1, the number of states in **VECTORMATON** is bounded by $O(m)$, and by Lemma 2, the total size of all ID sets is bounded by $O(m^{1.5})$. If the symbol set size is constant, each state has at most a constant number of outgoing transitions. Therefore, the total number of transitions is also bounded by $O(m)$, and the overall space consumption is dominated by the storage of ID sets and associated index structures, which is bounded by $O(m^{1.5})$.

I DELETION OF VECTORMATON

We discuss a lazy-deletion strategy that supports moderate deletion workloads. To enable deletions, we maintain, for each sequence, the set of ESAM states in which it appears. This information can be collected naturally during index construction and insertion, as each ESAM state already maintains the IDs of its associated sequences. To delete a sequence-vector pair with ID i , we first identify all states associated with the sequence using this auxiliary mapping. For each such state, we mark ID i as deleted in the corresponding HNSW index without physically removing it from the graph structure. During query processing, any deleted IDs returned by HNSW search are filtered out before producing the final results.

This approach avoids expensive structural modifications to the HNSW indexes, and similar lazy-deletion mechanisms have been adopted in prior work on HNSW-based indexes (e.g., [25]). The trade-off is that deleted entries continue to occupy index space and may slightly degrade query performance as their number grows. Therefore, this strategy is most suitable for workloads with infrequent or moderate deletion rates.

J DATASET DESCRIPTIONS

The detailed descriptions are provided below: (1) **Spam** [47]: a dataset of spam emails from the SpamAssassin corpus. Each record consists of an email title used as the sequence, and a pretrained embedding of the email content (generated by `all-MiniLM-L6-v2`) as the vector; (2) **Words** [15]: a dataset of word embeddings. Each record consists of a word (a sequence of letters) as the sequence and its pretrained word embedding as the vector; (3) **MTG** [49]: a dataset of image embeddings. Each record consists of an image description as the sequence and a pretrained image embedding as its vector; (4) **ArXiv** [37]: a dataset of paper titles and embeddings. Each record consists of a paper title as the sequence and a text embedding (generated by `all-mpnet-base-v2`) as its vector; (5) **SwissProt** [28]: a subset of the UniProt protein sequence database. Each record consists of a protein sequence as the sequence and its structural embedding (generated by `ProtBERT`) as the vector; (6) **CodeSearchNet** [23]: a dataset of code snippets. Each record consists of a function name as the sequence and a code embedding (generated by `CodeBERT`) as its vector.

K QUERY PERFORMANCE WITH LONGER QUERY PATTERNS

Figure 16 presents the complete results for longer query patterns with $|p| \in \{5, 6, 7\}$ across all datasets, extending the ArXiv results shown in Figure 9 in the main paper. In some subplots, `PostFiltering` and `ACORN` results are omitted as their recalls are consistently less than 0.1. As the pattern length increases, the query selectivity decreases substantially, making the workloads increasingly challenging for post-filtering-based methods. Consequently, `PostFiltering` and `ACORN` generally exhibit significant recall degradation. Although `PreFiltering` benefits from the reduced candidate set size and achieves higher query efficiency than in the short-pattern setting, it remains substantially slower than **VECTORMATON**. Overall, **VECTORMATON** consistently maintains high recall while achieving superior query throughput across all datasets and pattern lengths.

L EFFECTS OF PARAMETER T

Figure ?? illustrates the impact of varying T , where the queries are generated with mixed pattern lengths $|p| \in \{2, 3, 4\}$. As T increases, both index size and construction time decrease, since more states skip graph construction. While increasing T degrades query performance, query performance remains stable within a moderate range of T , indicating that brute-force search over small vector sets is sufficiently efficient. This suggests a trade-off between index construction cost and query efficiency when selecting T .

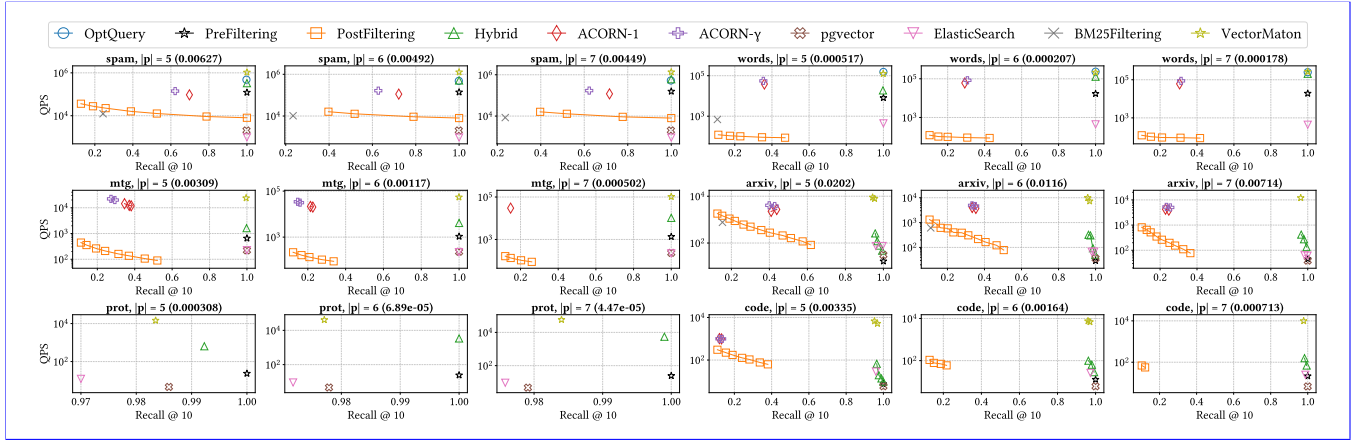


Figure 16: Query performance on longer patterns $|p| \in \{5, 6, 7\}$, where (x) in the title of each subplot denotes the average query selectivity is x .

